

Actividad Práctica

PONDERACIÓN: 100 pts

Tiempo estimado: 2 hrs

Universidad San Carlos de Guatemala
Facultad de Ingeniería.
Escuela de Ingeniería en Ciencias y Sistemas

Índice

1. MARCO FORMATIVO	2
1.1. Valores	2
1.2. Competencia(s)	2
1.3. Habilidad(es) blandas a formar	2
2. Resultado del Aprendizaje	2
2.1 Contenido	2
3. Actividad a desarrollar	3
3.1. Herramientas	3
3.2. Descripción de la actividad	3
3.3. Material de apoyo (referencias)	3
4. Rúbrica de Calificación	3
Requisitos para optar a la calificación	3
Detalle de la Calificación	3

1. MARCO FORMATIVO

1.1. Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
responsabilidad	desarrollar y probar correctamente las peticiones web, validando que la información enviada y recibida en formato JSON sea coherente y segura

1.2. Competencia(s)

Con la elaboración de esta actividad usted adquirirá la siguiente competencia:

- Demuestra destreza y habilidad en la selección, uso y adaptación de herramientas tecnológicas para desarrollar aplicaciones web que intercambian datos mediante JSON, consumen APIs REST con HttpClient y aplican respuestas HTTP adecuadas en ASP.NET Core.

1.3. Habilidad(es) blandas a formar

La actividad le permitirá desarrollar las siguientes habilidades:

- Colaborar con los compañeros en la resolución de errores relacionados con serialización, consumo de APIs y códigos de respuesta HTTP, fortaleciendo la comunicación técnica y el trabajo en equipo.

2. Resultado del Aprendizaje

2.1 Contenido

A continuación se detalla el contenido temático que será abordado en la actividad de laboratorio.

No.	Unidad	Tema	Sub Tema
1	JSON en C# y ASP.NET Core	JavaScript Object Notation	Concepto de JSON y uso en C#
2	JSON en C# y ASP.NET Core	JSON vs XML en .NET	Comparativa práctica y casos de uso



No.	Unidad	Tema	Sub Tema
3	JSON en C# y ASP.NET Core	Tipos de datos JSON	Mapeo entre JSON y tipos C#
4	JSON en C# y ASP.NET Core	Serialización y deserialización	System.Text.Json y Newtonsoft.Json
5	Acceso a Datos Web con C#	Internet, Web y HTTP	Modelo cliente-servidor y HttpClient
6	Acceso a Datos Web con C#	Códigos de respuesta HTTP	Estados 2XX, 4XX y 5XX en Web API
7	Integración Web	Razor Pages + Web API	Consumo de JSON desde frontend a backend

3. Actividad a desarrollar

3.1. Herramientas

Visual Studio Code, Visual Studio, Rider, .NET SDK, ASP.NET Core Web API, Razor Pages, navegador web y Postman o herramienta equivalente para probar solicitudes HTTP.

3.2. Descripción de la actividad

Problema: Consumo y Gestión de Productos con JSON en ASP.NET Core

Desarrolle una aplicación web utilizando ASP.NET Core Web API y Razor Pages que permita administrar y consultar un catálogo de productos mediante intercambio de datos en formato JSON. El objetivo es aplicar los temas vistos en la semana 11 sobre JSON, serialización, deserialización, HttpClient, códigos de respuesta HTTP e integración cliente-servidor.

El backend debe exponer una API REST para gestionar productos. Cada producto debe contar como mínimo con identificador, nombre, descripción, precio, disponibilidad y categoría. La información puede almacenarse en memoria mediante una lista para facilitar el desarrollo de la práctica.

La API debe devolver respuestas en formato JSON utilizando los mecanismos de serialización de ASP.NET Core. Se debe emplear System.Text.Json o Newtonsoft.Json para convertir objetos C# a JSON y para interpretar datos JSON recibidos desde las solicitudes del cliente.

Implemente endpoints que permitan listar todos los productos, consultar un producto por identificador, agregar un nuevo producto, actualizar información existente y eliminar un registro. Para ello se deben utilizar los métodos HTTP GET, POST, PUT o PATCH y DELETE según corresponda.

El sistema debe manejar correctamente los códigos de respuesta HTTP. Cuando una operación sea exitosa debe utilizar respuestas como Ok(), Created() o NoContent(). Cuando existan errores de datos o recursos inexistentes debe responder con BadRequest(), NotFound() u otro código adecuado.

Desarrolle una página Razor Pages que consuma la Web API mediante HttpClient. La página debe solicitar los productos al backend, deserializar la respuesta JSON y mostrar la información al usuario de forma clara. También puede incluir un formulario sencillo para enviar datos al API.

La solución debe evidenciar la comunicación cliente-servidor: Razor Pages actuará como frontend, la Web API como backend y JSON como formato de intercambio. Se recomienda mantener una estructura organizada mediante modelos, servicios y controladores.

Al finalizar, el estudiante deberá probar los endpoints con Postman o herramienta equivalente y verificar que el frontend consume correctamente los datos. Se espera claridad en el código, nombres adecuados, validaciones básicas, manejo de errores y uso correcto de JSON.

3.3. Material de apoyo (referencias)

Agregar documentación oficial de Microsoft sobre System.Text.Json, ASP.NET Core Web API, Razor Pages, HttpClient, códigos de estado HTTP y material de clase de la semana 11.

4. Rúbrica de Calificación

Requisitos para optar a la calificación

* La actividad debe realizarse y ser entregada durante el periodo de clase.

Detalle de la Calificación

Criterio	Descripción	Puntos Máximos
Uso de JSON y modelos C#	Se evalúa la definición de modelos, tipos de datos JSON y serialización/deserialización con System.Text.Json o Newtonsoft.Json.	25
Implementación de Web API	Se evalúa la creación de endpoints REST y el uso correcto de GET, POST, PUT/PATCH y DELETE en ASP.NET Core.	25
Consumo con Razor Pages	Se evalúa el consumo de la API mediante HttpClient, la lectura de respuestas JSON y la presentación de datos en el frontend.	25



Códigos HTTP y validaciones	Se evalúa el uso correcto de respuestas 2XX, 4XX y 5XX, validaciones básicas, manejo de errores y pruebas con Postman.	25
Total		100

